

HAVEN

Rapport Technique Officiel — V2 (corrections post-bêta Android réelle)

Suite à une première bêta réelle sur téléphone Android de HAVEN V1, cette V2 corrige les problèmes observés (animation, rythme de démarrage, lisibilité des améliorations) sans ajouter de contenu ni de Map 2, sans toucher à la monétisation. Dépôt : [scuillerjack-oss/haven-v0](https://github.com/scuillerjack-oss/haven-v0) · Branche : main.

Convention de ce rapport : **FAIT** = codé. **TESTÉ** = vérifié par un test automatisé (moteur ou navigateur headless). **DÉPLOYÉ** = le workflow GitHub Actions de déploiement a réussi sur le commit concerné. **NON VÉRIFIÉ** = pas encore confronté à un vrai appareil ou une vraie condition réseau depuis cette session. **REPORTÉ** = décidé explicitement de ne pas construire maintenant. Aucun comportement n'est déclaré fonctionnel sur un véritable téléphone s'il n'a été vérifié qu'en environnement automatisé.

0. État initial réellement trouvé (audit avant toute modification)

Le cahier des charges V2 référence le commit **b36df5e** comme dernier commit de la mission V1. Vérification réelle par git log avant toute modification : ce n'est PAS le dernier commit du dépôt. 3 commits de documentation existaient déjà après lui, sans changement fonctionnel :

- **33decd6** — Ajoute le rapport technique officiel V1 (PDF)
- **1b60c49** — Corrige une surestimation dans le rapport V1 (l'URL live n'avait en fait pas pu être revérifiée depuis la session — même limite réseau que celle documentée ci-dessous, section 11)
- **96e3d70** — Met à jour le README pour la V1

État réel de départ de cette mission V2 : commit **96e3d70**, branche **main**, dernier déploiement GitHub Actions vert sur ce commit (run #9, build+test+test:mobile+deploy). Tous les acquis V1 listés dans le cahier des charges (moteur économique séparé, chaîne production→stockage→transport, goulots d'étranglement, sauvegarde/migration, progression hors-ligne, Perles, Renaissance, atouts permanents, onboarding, architecture data-driven multi-map, correctifs mobile, tests, Pages, CI) étaient bien présents et déployés à ce commit — vérifié par lecture directe du code, pas supposé depuis le rapport V1 seul.

1. Commits créés pendant cette mission V2

Commit	Jalon	Contenu
54a1fad	1	Animation du travailleur : jambes/bras détachés, téléportation à l'arrêt
3010ada	2	Courbe de coûts/valeurs : début rapide + durée totale non raccourcie
039fb2f	3	Cartes d'amélioration : grandeur concrète valeur actuelle → suivante
9b7d9ec	4	Tests de non-régression permanents (rythme + durée totale)

Chaque commit a été poussé séparément vers origin/main immédiatement après ses propres tests verts (jamais un commit unique fourre-tout).

2. Problèmes révélés par la bêta Android réelle, et correctifs apportés

2.1 Jambes détachées pendant la marche (priorité absolue)

Cause racine exacte, trouvée par audit de code puis confirmée visuellement (Playwright headless) : un attribut SVG `transform="translate(...)"` statique est entièrement ÉCRASÉ (pas combiné) par toute transformation CSS animée appliquée au MÊME élément. Les jambes/bras, des `<line>` tournées par CSS sans `transform-origin` explicite, pivotaient donc autour du centre de leur propre boîte englobante plutôt que de la hanche/l'épaule — l'origine exacte de l'impression de membres détachés.

Corrigé en séparant strictement, pour chaque articulation : un groupe PARENT statique (positionnement par attribut, jamais animé) et un groupe ENFANT sans attribut de transformation, recevant l'animation CSS avec `transform-box: view-box; transform-origin: 0px 0px;` — qui coïncide alors exactement avec l'articulation réelle. Le même bug affectait aussi, de façon indépendante, la bulle d'attente "stockage plein" (son animation l'envoyait

invisible à l'origine du SVG) — trouvé en vérifiant visuellement le correctif de téléportation ci-dessous, pas seulement le rapport initial du problème.

2.2 Téléportation à l'arrêt (stockage plein)

Le moteur V1 ne détectait la saturation du stockage qu'à la toute fin du cycle complet du travailleur, après un trajet retour déjà animé comme réussi — puis affichait le travailleur téléporté à un point arbitraire proche du stockage. Le cycle est désormais scindé en deux étapes explicites dans mapEconomy.js : outbound (préparation → arrivée au stockage) et return (retour au puits). La livraison est tentée exactement à la frontière entre les deux ; si la place manque, le travailleur se fige pile à l'arrivée au stockage (producer.awaitingRoom = true), jamais après un retour à vide animé pour rien.

2.3 Début de partie trop lent (priorité absolue)

En V1, le joueur restait quasi-spectateur : sur le profil « léger » (2 sessions/jour de 1 min), la toute première amélioration de la chaîne n'était accessible qu'après **~12 heures** de jeu réel écoulé (43 201 000 ms, mesuré par simulation avec le vrai moteur, pas estimé). Sur le profil « modéré », seules 2 familles sur 8 étaient accessibles avant 8 heures. Cause : le coût de base des familles non plafonnées restait élevé relativement au revenu de départ, et l'intervalle de transport de base (60s) limitait le premier revenu perceptible. Corrigé par une recalibration complète de la courbe (section 4).

3. Nouvelle économie des premières minutes

Recalibration par simulation (jamais par un simple multiplicateur global appliqué à tout, qui aurait seulement déplacé le problème) : chaque famille non plafonnée (seau, stockage, citerne, productivité) utilise désormais une courbe géométrique plus prudente (12 niveaux au lieu de 16, costGrowth ~2.1–2.45, valueGrowth 1.25 au lieu de 1.4/16 niveaux) et l'intervalle de transport de base est descendu de 60s à 12s. Premier achat mesuré : **24 secondes** de jeu actif (objectif du cahier des charges : 20-40s). Table avant/après, premier achat de CHAQUE famille, mesuré par le même moteur réel :

Famille	Léger — V1	Léger — V2	Modéré — V1	Modéré — V2	Assidu — V1	Assidu — V2
Seau	~12h	24s	2 min	24s	2 min	24s
Déplacement	~12h	48s	3 min	48s	3 min	48s
Corde/treuil	~12h	~12h*	8h	1 min 47s	4 min	1 min 47s
Puits	~12h	~12h*	8h	2 min 54s	5 min	2 min 54s
Stockage	~12h	~12h*	8h	2 min 20s	8h	2 min 20s
Citerne	~12h	~12h*	8h	8h	8h	4 min 12s
Fréq. transport	~12h	~12h*	8h	1 min 24s	8h	1 min 24s
Productivité	~12h	~12h*	8h	8h	~4.8h	~4.8h

* Profil « léger » (2 sessions/jour de 1 min) : seuls seau et déplacement passent sous la minute de jeu actif cumulé ; les familles plus chères restent, y compris en V2, gagnées après plusieurs heures — c'est voulu (« espacement progressivement plus grand » demandé par le cahier des charges), pas un oubli.

Décisions cumulées mesurées dans les 3 premières minutes de jeu actif continu : **8 achats** (contre 2 en V1 sur la même fenêtre, tous deux mesurés par le même moteur). Détail complet (checkpoints 1/3/10/30/60 min, profils hors-ligne) dans docs/haven-v2-simulation-results.json, reproductible via node scripts/simulate-v2.mjs.

4. Tableau récapitulatif avant/après (V1 → V2)

Mesure	V1	V2
Premier achat possible (jeu actif continu)	Variable, jusqu'à ~12h selon profil	24s
Achats cumulés en 3 min de jeu actif	2	8

Mesure	V1	V2
Map 1 complète — profil léger (calendaire / actif)	3.5 j / 7.0 min	3.5 j (3j12h) / 7.0 min
Map 1 complète — profil modéré (calendaire / actif)	2.0 j / 18.0 min	2.33 j (2j8h) / 21.0 min
Map 1 complète — profil assidu (calendaire / actif)	1.2 j / 30.0 min	1.375 j (1j9h) / 35.0 min
Plancher visuel des phases animées	Non garanti (bug trouvé, voir §6)	≥ 250ms, garanti par test
Plancher visuel du transport	60s de base, pas de plancher explicite	≥ 6000ms, garanti par test
Jambes/bras pendant la marche	Se détachent visuellement (bêta réelle)	Restent attachés (vérifié Playwright)
Arrêt sur stockage plein	Téléportation après un retour animé pour rien	Arrêt net au point de livraison
Affichage d'une amélioration	Nom + prix seulement	Niveau $X \rightarrow Y$ + grandeur réelle $X \rightarrow Y$
Tests moteur automatisés	38	48

Lecture importante : la durée totale (calendaire ET active) n'a jamais été raccourcie — elle est égale ou légèrement supérieure à la V1 sur les 3 profils, alors que le début est désormais nettement plus rapide. Ce n'était pas automatique : une première tentative de recalibration (documentée en section 5) avait accéléré le début mais fait s'effondrer la durée totale à 2-10 minutes actives — détecté et corrigé avant tout commit, par la même exigence de simulation que celle qui avait détecté le problème inverse en V1.

5. Stratégie contre le problème de « vitesse infinie » (section 4 du cahier des charges)

Séparation stricte, dès la définition des familles (mapDefinitions.js), entre deux formes de courbes :

- **Valeur croissante sans plafond** (seau, stockage, citerne, productivité) : chaque niveau coûte plus cher ET rapporte plus, géométriquement — c'est l'axe de progression long terme, jamais limité par la lisibilité de l'animation puisqu'aucune de ces familles n'accélère une animation.
- **Durée décroissante VERS UN PLAFOND** (déplacement, treuil, puits, fréquence de transport) : le multiplicateur de temps descend linéairement vers un plancher DÉRIVÉ mathématiquement de $\text{MIN_VISUAL_PHASE_MS}$ (250ms) / $\text{MIN_VISUAL_TRANSPORT_INTERVAL_MS}$ (6000ms), jamais fixé à la main — le plancher de chaque famille est `plancher_visuel / durée_de_la_phase_la_plus_courte_affectée`.

Bug réel trouvé par simulation avant tout déploiement : le plancher lui-même était arrondi « au plus proche » (`Math.round`) plutôt que « vers le haut ». Pour le puits (2400ms de base), $250/2400 = 0.10417$ arrondissait à 0.10, ce qui repassait sous le plancher au dernier niveau (240ms au lieu de 250ms minimum). Corrigé en remplaçant l'arrondi par un arrondi vers le haut (`ceil2`) pour les 3 planchers dérivés — garanti maintenant par un test permanent (`tests/mapDefinitions.test.js`) qui vérifie, pour CHAQUE niveau de CHAQUE famille plafonnée, qu'aucune durée ne descend sous son plancher.

Une fois le plancher atteint (dernier niveau de chaque famille plafonnée), l'axe est explicitement épuisé : toute progression économique ultérieure doit venir des familles non plafonnées ou de futurs paliers qualitatifs — l'architecture ne verrouille jamais un travailleur à une vitesse visuelle absurde, quel que soit le niveau atteint dans une future V3 avec beaucoup plus de niveaux.

6. Valeurs exactes des améliorations (extraites du moteur réel, pas recopiées à la main)

Exemple de rendu réel sur une carte d'amélioration (vérifié en navigateur, Playwright) :

- Seau — Niveau 1 $\rightarrow$ 2 / 1 L $\rightarrow$ 1.3 L par trajet

- Déplacement — Niveau 1 → 2 / 1.8s → 1.6s
- Corde / treuil — Niveau 1 → 2 / 1.2s → 1s
- Puits — Niveau 1 → 2 / 2.4s → 2.1s

Table complète des 4 premiers niveaux et du niveau maximum de chaque famille (WATER_MAP réelle) :

Famille	Niv. 1	Niv. 2	Niv. 3	Niv. 4	Niv. max (coût)
Seau (L/trajet)	1 (0)	1.25 (3)	1.56 (7)	1.95 (18)	11.64 @12 (23 377)
Déplacement (mult.)	1.00 (0)	0.89 (4)	0.77 (7)	0.66 (12)	0.21 @8 (97)
Corde/treuil (mult.)	1.00 (0)	0.87 (6)	0.74 (11)	0.61 (18)	0.21 @7 (98)
Puits (mult.)	1.00 (0)	0.87 (8)	0.75 (14)	0.62 (26)	0.11 @8 (272)
Stockage (L)	20 (0)	25 (6)	31.25 (14)	39.06 (35)	232.83 @12 (38 042)
Citerne (L)	20 (0)	25 (8)	31.25 (20)	39.06 (48)	232.83 @12 (62 338)
Fréq. transport (s)	12.0 (0)	11.1 (5)	10.3 (9)	9.4 (14)	6.0 @8 (121)
Productivité (mult.)	1.00 (0)	1.15 (60)	1.32 (144)	1.52 (346)	2.66 @8 (11 466)

Format : valeur (coût en argent). Table extraite automatiquement de WATER_MAP, jamais recopiée à la main — élimine le risque de divergence entre le rapport et le code réel.

Conformité vérifiée par test permanent (tests/upgrades.test.js) : pour chaque famille, un achat réel est effectué puis la grandeur affichée est comparée à ce que mapEconomy.js applique réellement (valeur relue au moteur après achat, jamais une formule dupliquée côté UI qui pourrait diverger silencieusement).

7. Simulations économiques (reproductibles, jamais estimées à la main)

node scripts/simulate-v2.mjs — résultats dans docs/haven-v2-simulation-results.json. Couvre explicitement les fenêtres demandées (1/3/10/30/60 min de jeu actif continu) et les 3 profils de session avec hors-ligne plafonné comme un vrai retour (jamais un tick continu déguisé).

Jeu actif continu	Achats cumulés	Argent	% de la map
1 min	2	4.5	0.00%
3 min	8	1.7	0.00%
10 min	20	12.1	0.02%
30 min	40	67.6	0.13%
1 h	53	567.7	0.61%

Map 1 complète en jeu actif continu (jamais fermé, cas extrême et peu réaliste, mesuré pour vérifier l'absence de point mort économique) : **~41h** (147 781 000 ms), 67 achats au total — confirme qu'il n'y a pas de blocage économique, seulement une progression qui ralentit naturellement une fois toutes les familles achetées, comme attendu d'un idle.

Profils réalistes avec hors-ligne plafonné (identique méthodologie qu'en V1, pour rester comparable) :

Profil	Map 1 complète (calendaire)	dont temps ACTIF	Renaissance dispo.	Achats totaux
Léger (2×1min/j)	3j 12h	7 min	1 j	67
Modéré (3×3min/j)	2j 8h	21 min	16 h	67
Assidu (5×5min/j)	1j 9h	35 min	9h 36min	67

Plafonds de vitesse visuelle vérifiés par le même script : toutes les phases ≥ 250 ms (pire cas : déplacement/retour niveau 8 $\rightarrow$ 252ms), tous les intervalles de transport ≥ 6000 ms (pire cas : niveau 8 $\rightarrow$ 6000ms pile, comme attendu d'un plancher atteint exactement).

8. Tests exécutés et résultats exacts

48 tests node:test (0 échec, 0.3s) + suite Playwright mobile (5/5 largeurs, 0 échec) :

Fichier	Nb tests	Couvre
economy.test.js	13	Production, coûts, goulots, saturation, hors-ligne, dérive numérique
mapDefinitions.test.js	4	NOUVEAU V2 — planchers de vitesse visuelle, croissance stricte des coûts
meta.test.js	6	Perles, Renaissance, atouts (aucune régression)
migrations.test.js	7	Migration V0 $\rightarrow$ V1 ET V1 $\rightarrow$ V2 (nouveau modèle outbound/return), champs absents
pacing.test.js	5	NOUVEAU V2 — premier achat ≤ 40 s, plusieurs décisions en 3 min, durée totale non raccourcie
save.test.js	5	Sauvegarde/rechargement, aucune perte
simulation.test.js	3	Intégration tick + hors-ligne, déblocage de map
upgrades.test.js	5	Achats, refus, réduction de coût, NOUVEAU V2 : conformité valeur affichée $\leftrightarrow$ moteur

Suite mobile (npm run test:mobile, Playwright headless réel, pas une simulation) :

- Clic fonctionnel après suspend/resume simulé — toujours vert après la refonte du cycle producteur
- Aucune erreur console pendant le cycle suspend/resume
- Crédit hors-ligne idempotent malgré des signaux de reprise redondants (pas de double crédit)
- Aucun débordement horizontal à 320/360/390/414/480px — vérifié après l'ajout des lignes « Niveau X $\rightarrow$ Y » et « grandeur d'effet » sur chaque carte

Build de production : npm run build propre (dist/index.html 0.75kB, CSS 7.65kB, JS 37.14kB gzip 12.23kB), aucune erreur.

9. État réel du déploiement

4 push sur main pendant cette mission, chacun avec le workflow « Deploy to GitHub Pages » (build + npm test + npm run build + Playwright + test:mobile + deploy-pages) — vérifié via l'API GitHub Actions, pas supposé. Le job de build a été annulé sur 2 commits intermédiaires par la politique de concurrence du workflow (cancel-in-progress, un push plus récent arrivant avant la fin du précédent) : c'est le comportement voulu, seul le DERNIER commit compte pour l'état déployé.

État réel confirmé sur le dernier commit (**9b7d9ec**, run #13) : job build réussi (npm ci, npm test, npm run build, playwright install, npm run test:mobile — chaque étape verte individuellement) ET job deploy réussi (actions/deploy-pages@v4). URL inchangée : <https://scuillerjack-oss.github.io/haven-v0/>.

Limite honnête, identique à celle déjà documentée en V1 : cette session tourne dans un environnement dont la politique réseau bloque les domaines externes non listés, y compris github.io (confirmé de nouveau par un test direct : connexion rejetée par le proxy réseau). Je n'ai donc pas pu recharger moi-même l'URL publiée pour la revérifier visuellement depuis cette session — seulement confirmer, via l'API GitHub, que le job de déploiement a réellement réussi sur le commit final. Un chargement réel sur un téléphone reste la vérification qui compte et n'a pas été refaite pour cette V2 précise : **NON VÉRIFIÉ**.

10. Limites connues et éléments non vérifiés ou reportés

- **NON VÉRIFIÉ** — Correctifs d'animation (jambes/bras, arrêt net) confirmés en Playwright headless (Chromium), PAS encore sur l'appareil Android réel où le bug d'origine a été observé. C'est la vérification qui compte et n'a pas pu être refaite depuis cette session.
- **NON VÉRIFIÉ** — Nouveau rythme de démarrage (24s, 8 achats en 3 min) confirmé par simulation avec le vrai moteur, PAS encore par un nouveau test humain réel sur le terrain.
- **REPORTÉ** — Aucune Map 2 construite (hors périmètre explicite de cette V2).
- **REPORTÉ** — Aucune interface de monétisation (hors périmètre explicite de cette V2, décision de prudence maintenue).
- Les familles les plus chères (citerne, productivité) restent volontairement gagnées après plusieurs heures même sur le profil assidu — c'est l'« espacement progressivement plus grand » demandé, pas une régression du début rapide.
- Art de la scène toujours minimal (formes géométriques simples), priorité donnée à la lisibilité de l'animation et à la performance — inchangé depuis la V1, non demandé en V2.

11. Recommandations pour la V3

- Confronter les 3 correctifs de cette V2 (animation, démarrage, lisibilité) à un vrai test humain sur le même appareil Android que la première bêta, avant toute nouvelle fonctionnalité — c'est la seule vérification qui n'a pas pu être faite depuis cette session.
- Si le test humain confirme le nouveau rythme de démarrage, envisager de resserrer légèrement l'écart entre profils « léger » et « assidu » sur les familles chères (citerne, productivité), qui restent actuellement à plusieurs heures même pour un joueur assidu.
- Construire une Map 2 réelle sur l'architecture déjà prête (mapEconomy.js/mapUpgrades.js réutilisés sans duplication), seulement une fois le rythme de démarrage validé humainement.
- Étudier les interfaces rewarded/premium seulement après ce test humain — pas avant, comme le cahier des charges le demande explicitement depuis la V1.

- Si une V3 ajoute des niveaux au-delà des plafonds de vitesse actuels, faire venir la progression économique de nouvelles familles non plafonnées (nouveaux postes, nouveau matériel, paliers qualitatifs) plutôt que de relâcher les planchers visuels existants.

Fin du rapport — HAVEN V2, dépôt scuillerjack-oss/haven-v0, branche main.